

CYBERSECURITY PROJECT REPORT

Automated Threat Intelligence Collection and AI Assisted Briefing

A Python Pipeline Using Public Feeds and a Locally Hosted Language Model

Quinn Smith
University Of Wisconsin Platteville

Executive Summary

Security teams cannot read everything that is published about threats. Each day brings vulnerability announcements, malware reports, and news articles, and only a few of them matter to a particular environment. This project is an automated threat intelligence pipeline written in Python that collects data from the CISA Known Exploited Vulnerabilities catalog and three security news feeds, stores the items in a SQLite database, extracts indicators of compromise with deterministic regular expressions, and uses a locally hosted large language model to produce a prioritized daily briefing.

The design separates tasks that must be exact, such as indicator extraction, from tasks where a language model adds value, such as summarization and prioritization. Running the model locally through Ollama removes API costs and keeps collected data on the analyst's own machine.

Key finding: In a test run on September 30, 2026, the pipeline collected 57 new items and summarized 25 of them in five batches with no failed batches. This was a functional test and not an accuracy evaluation, and the report states the limits of what it shows.

The report also examines the security risks of feeding untrusted text to a language model. The OWASP project ranks prompt injection as the first risk for applications built on language models (OWASP Gen AI Security Project, 2025), and research on indirect prompt injection shows that instructions hidden in external content can manipulate a model (Greshake et al., 2023). The pipeline reduces these risks by keeping the model away from indicator values, labeling feed content as data, and giving the model no tools to act with.

1. Introduction

Threat intelligence is information about threats and threat actors that helps an organization defend itself. Surveys of the field describe a large and growing set of sources, including vendor reports, government advisories, community feeds, and news, and these sources vary widely in quality and format (Tounsi & Rais, 2018). Guidance from the National Institute of Standards and Technology explains that organizations benefit from sharing and consuming this information, but doing so requires the capacity to process it (Johnson et al., 2016). For a small team, or for one person learning the field, the practical problem is attention. There are more reports each day than anyone can read.

This project addresses that problem with automation. The goal was to build a working system that gathers information each day, removes duplicates, pulls out concrete indicators, and uses a language model to write a short briefing ranked by severity and relevance. I also wanted the project to demonstrate sound engineering judgment about where a language model belongs in a security workflow and where it does not.

2. Background

2.1 Known Exploited Vulnerabilities

The Cybersecurity and Infrastructure Security Agency (CISA) maintains a catalog of vulnerabilities that are known to have been exploited in real attacks. The catalog supports Binding Operational Directive 22 01, which requires federal civilian agencies to remediate listed vulnerabilities by set due dates (CISA, 2021). The catalog is published as a JSON file, which makes it a reliable and machine readable starting point for a

pipeline (CISA, n.d.). An entry in this list carries more weight than an ordinary vulnerability announcement because it indicates that attackers are already using the flaw.

2.2 Vulnerability Scoring and Enrichment

The National Vulnerability Database, run by NIST, provides descriptions and severity scores for published CVE records (NIST, n.d.). Severity alone does not show how likely a flaw is to be attacked. The Exploit Prediction Scoring System (EPSS) addresses this by estimating the probability that a vulnerability will be exploited in the wild, using observed data rather than only technical severity (Jacobs et al., 2021). This project currently uses the exploitation status from the CISA catalog, and EPSS and NVD enrichment are planned as future work.

2.3 MITRE ATT&CK

MITRE ATT&CK is a knowledge base of adversary tactics and techniques built from observed behavior, designed to give defenders a shared vocabulary (Strom et al., 2018). Mapping a report to ATT&CK technique identifiers helps an analyst connect a news item to detection and mitigation work. In this project the language model is allowed to suggest technique identifiers, which makes validation of those suggestions an important concern discussed later.

2.4 Language Models in Security Workflows

Large language models are good at condensing text and rewriting it for a target reader, which suits the task of summarizing many articles. They also have known weaknesses. Models can produce fluent text that is not supported by the source material, a problem the literature calls hallucination (Ji et al., 2023). Models that read external text can also be manipulated by instructions hidden inside that text, an attack known as indirect prompt injection (Greshake et al., 2023; OWASP Gen AI Security Project, 2025). Both weaknesses shaped the design of this project.

To run a model locally, I used Ollama, a tool that downloads open weight models and serves them through a local REST interface (Ollama, n.d.). The model used in testing was Llama 3.1 with eight billion parameters, part of the Llama 3 family of open models (Grattafiori et al., 2024).

3. Objectives and Threat Model

3.1 Project Objectives

- Collect threat information from public sources automatically and store it in a structured form.
- Avoid processing the same item twice across repeated runs.
- Extract indicators of compromise exactly as they appear in the source, with no risk of invented values.
- Produce a short briefing that ranks items by severity and flags those relevant to a defined technology environment.
- Run at no cost and without sending collected data to a third party service.

3.2 Threat Model

The relevant attacker is assumed to be able to publish content that the pipeline will collect, such as a blog post, a news article, or a malware description. The attacker has no access to the analyst's computer and no

control over the code. Instead, the attacker tries to exploit the fact that the pipeline passes outside text to a language model.

- Publish text containing instructions aimed at a language model that reads it.
- Write misleading claims intended to raise or lower the apparent severity of an item.
- Include false or misleading indicators in article text.
- Produce large numbers of similar items that compete for the limited number of items summarized in each run.

The security objectives are the integrity of the briefing, the confidentiality of the description of the analyst's environment, and the continued operation of the pipeline when one input is malformed.

4. System Design

The system is a linear pipeline with six stages. Each stage has a single responsibility, which makes the program easy to test and extend. Table 1 summarizes the stages.

Stage	Purpose	Implementation
Collect	Retrieve new data from public feeds	Python requests library for the CISA JSON catalog and feedparser for RSS
Normalize	Convert every source into one schema	Fields for source, title, link, date, and text
Deduplicate	Skip items already seen	Hashed item identifier used as a SQLite primary key
Extract	Find indicators of compromise	Regular expressions, with no model involvement
Summarize	Rank items and write the briefing	Local Llama 3.1 model through Ollama, returning JSON
Deliver	Present the result to the analyst	Markdown file sorted by severity

Table 1. Pipeline stages and their implementation.

The central design principle is a division of labor. Anything that must be exact is handled by ordinary code. Anything that benefits from language understanding is handled by the model. Indicators such as IP addresses and file hashes must be reproduced exactly, so the model never produces or edits them. The model is responsible for summaries, severity labels, technique suggestions, recommended actions, and a judgment about relevance.

5. Implementation

5.1 Collection and Storage

The collector reads the CISA catalog and keeps entries added within the previous 14 days, which prevents the first run from loading the entire historical catalog. It also reads three security news feeds (BleepingComputer, The Hacker News, and Krebs on Security) using the feedparser library (feedparser, n.d.). Up to 15 recent entries are taken from each feed, and HTML markup is removed from article summaries.

All items are stored in a SQLite database through Python's built in sqlite3 module (Python Software Foundation, n.d.). SQLite was chosen because it needs no server and stores everything in a single file. Each item receives a 16 character identifier made from a SHA256 hash of its source and unique key. Because this identifier is the primary key, inserting an item that already exists raises an integrity error that the program

treats as a signal to skip it. A “digested” flag records which items have already appeared in a briefing, so repeated runs only summarize new material.

5.2 Indicator Extraction

Indicators are extracted with regular expressions for the types shown in Table 2. The extractor first converts common defanged forms, such as bracketed dots and the prefix hxxp, back into normal form so that indicators written safely in articles are still found. Results are deduplicated and capped per type to keep the output readable.

Indicator type	Matching approach
CVE identifier	Pattern for the year and sequence number format
IPv4 address	Four octets, each validated to the range 0 through 255
File hashes	Hexadecimal strings of 32, 40, and 64 characters for MD5, SHA1, and SHA256
URL	Strings beginning with http or https
Domain	Hostnames ending in a limited list of common top level domains to reduce false matches

Table 2. Indicator types extracted by regular expression.

5.3 Summarization with a Local Model

Summarization uses the Ollama chat endpoint. The program sends items in batches of five, because smaller models follow instructions more reliably when given less text at once. Each article is truncated to 1,000 characters. The request sets a low temperature of 0.2 to reduce variation, a context window of 8,192 tokens, and the JSON output option so that the response can be parsed by code. If a batch fails, the program logs the error, skips that batch, and continues, so one malformed response does not end the run.

The system prompt tells the model to act as a threat intelligence analyst. It lists the technology environment of the reader, which in the current configuration includes Windows, Microsoft 365, Linux, Python, Apache, Cisco, and Fortinet, and asks the model to mark an item as relevant only if it plausibly affects that environment. The required output for each item is an identifier, a severity from critical to informational, a one or two sentence summary, a relevance flag, a list of ATT&CK technique identifiers, and a list of recommended actions.

5.4 Output

The final briefing is a Markdown file containing an overview and the items sorted from most to least severe. Each item shows its title, source, link, summary, techniques, and actions. The indicators shown under each item come from the regular expression extractor stored in the database, not from the model, and each item the model returns is matched back to a stored record by identifier. Any identifier the model returns that was not sent to it is ignored.

6. Security Considerations

6.1 Untrusted Input

Feed content is written by outside parties, and some of it, such as malware descriptions or blog posts, could be crafted by attackers. A pipeline that passes this text to a language model is exposed to indirect prompt injection, where hidden instructions in the data attempt to change the behavior of the model (Greshake et

al., 2023; OWASP Gen AI Security Project, 2025). The project reduces this risk in several ways. The instructions are placed in the system message and the feed data is wrapped in labeled tags, with an explicit rule that anything inside those tags is data and not instructions. The model has no tools and cannot take actions, so the worst outcome of a successful injection is a misleading summary rather than a harmful operation. The identifier check described above also prevents the model from adding records that were never collected. These measures reduce the risk but do not remove it, and the briefing should be read with that in mind.

6.2 Hallucination

Because a model can generate content that is not supported by its input (Ji et al., 2023), the project keeps the model away from the data where errors would be most damaging. Indicators are extracted by code. The prompt forbids inventing facts and instructs the model to cite an ATT&CK technique only when it is confident. A small model may still violate these instructions, which is why validating suggested technique identifiers against the official ATT&CK list is one of the planned improvements.

6.3 Privacy and Cost

Running the model locally means that collected material and the technology environment description are never sent to an external service. This matters in practice, because describing the software an organization runs is itself sensitive information. Local operation also removes per request fees, which made the project practical to build and run without a budget.

6.4 Secrets Handling

The current version uses no API keys. Future versions that call services such as NVD or abuse.ch should read keys from environment variables or a file excluded from version control, and should never place them in source code.

7. Testing and Observations

I ran the complete pipeline on a Windows machine on September 30, 2026. Table 3 shows the results of that run.

Measure	Result
New items from the CISA catalog	17
New items from the news feeds	40
Total new items collected	57
Items summarized in the run	25 (five batches of five)
Batch failures logged	0
Items left in the database for the next run	32

Table 3. Results of the test run on September 30, 2026.

The program wrote the briefing file at the end of the run, and the remaining 32 items stayed in the database marked as not yet digested. This confirmed that the collection, deduplication, extraction, summarization, and output stages work together from start to finish.

This was a functional test and not a formal accuracy evaluation. I did not measure how often the model assigned a correct severity or produced a correct technique identifier. The review process I use for each briefing is to check whether severity labels are consistent with the source, whether suggested ATT&CK identifiers exist and fit the described behavior, and whether each summary matches its title. Catalog entries should rarely be labeled low severity, since they describe vulnerabilities that are being exploited. A systematic evaluation is listed under future work.

8. Limitations

- Severity labels come from the model and not from measured data. Until EPSS and CVSS enrichment is added, a label is an estimate.
- A model with eight billion parameters is less capable than larger hosted models, and it may produce inconsistent JSON or shallow summaries. The batching and error handling reduce but do not eliminate this.
- The domain pattern uses a limited list of top level domains, so it will miss indicators with other endings, and any pattern based approach can match benign strings.
- Only one structured catalog and three news feeds are used, so coverage of malware and network indicators is narrow.
- The relevance flag depends on a short list of keywords describing the environment and on the judgment of the model.
- The output file is named by date, so a second run on the same day overwrites the first briefing.
- News articles about the same event from different outlets appear as separate items.

9. Future Work

- Add EPSS and NVD enrichment for every CVE and pass the scores to the model, so that severity is grounded in data (Jacobs et al., 2021; NIST, n.d.).
- Validate model suggested ATT&CK identifiers against the official knowledge base and discard any that do not exist (Strom et al., 2018).
- Add collectors for abuse.ch feeds such as URLhaus and ThreatFox to bring in malware and network indicators (abuse.ch, n.d.).
- Schedule daily runs with Windows Task Scheduler and deliver the briefing by email or a chat webhook.
- Build a labeled evaluation set and report agreement between model severity labels and reference sources, comparing several local models.
- Cluster near duplicate news items so one event produces one entry.
- Create a dashboard that lets an analyst search the database and filter by severity and relevance.

10. Conclusion

This project shows that a useful threat intelligence workflow can be built with free tools and a modest computer. Its main lesson is about boundaries. Deterministic code handles collection, storage, deduplication, and indicator extraction, where accuracy is essential, while a language model handles summarization and prioritization, where its strengths apply. Treating all feed content as untrusted, keeping

the model away from indicator values, and running everything locally address the most important risks of combining language models with security data. The system is a working foundation, and the planned enrichment, validation, and evaluation work would turn it from a functional prototype into a more dependable analyst tool.

References

- abuse.ch. (n.d.). *abuse.ch: Fighting malware and botnets*. <https://abuse.ch>
- Cybersecurity and Infrastructure Security Agency. (n.d.). *Known exploited vulnerabilities catalog*. <https://www.cisa.gov/known-exploited-vulnerabilities-catalog>
- Cybersecurity and Infrastructure Security Agency. (2021). *Binding operational directive 22 01: Reducing the significant risk of known exploited vulnerabilities*. <https://www.cisa.gov/news-events/directives/bod-22-01-reducing-significant-risk-known-exploited-vulnerabilities>
- feedparser. (n.d.). *feedparser documentation*. <https://feedparser.readthedocs.io>
- Grattafiori, A., Dubey, A., Jauhri, A., et al. (2024). The Llama 3 herd of models. *arXiv:2407.21783*. <https://arxiv.org/abs/2407.21783>
- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., & Fritz, M. (2023). Not what you've signed up for: Compromising real world LLM integrated applications with indirect prompt injection. *arXiv:2302.12173*. <https://arxiv.org/abs/2302.12173>
- Jacobs, J., Romanosky, S., Edwards, B., Adjerid, I., & Roytman, M. (2021). Exploit prediction scoring system (EPSS). *Digital Threats: Research and Practice*, 2(3). <https://doi.org/10.1145/3436242>
- Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., Ishii, E., Bang, Y., Madotto, A., & Fung, P. (2023). Survey of hallucination in natural language generation. *ACM Computing Surveys*, 55(12). <https://doi.org/10.1145/3571730>
- Johnson, C., Badger, M., Waltermire, D., Snyder, J., & Skorupka, C. (2016). *Guide to cyber threat information sharing* (NIST Special Publication 800 150). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-150>
- National Institute of Standards and Technology. (n.d.). *National vulnerability database*. <https://nvd.nist.gov>
- Ollama. (n.d.). *Ollama*. <https://ollama.com>
- OWASP Gen AI Security Project. (2025). *LLM01:2025 Prompt injection*. <https://genai.owasp.org/llmrisk/llm01-prompt-injection/>
- Python Software Foundation. (n.d.). *sqlite3: DB API 2.0 interface for SQLite databases*. <https://docs.python.org/3/library/sqlite3.html>
- Strom, B. E., Applebaum, A., Miller, D. P., Nickels, K. C., Pennington, A. G., & Thomas, C. B. (2018). *MITRE ATT&CK: Design and philosophy*. The MITRE Corporation. <https://attack.mitre.org>
- Tounsi, W., & Rais, H. (2018). A survey on technical threat intelligence in the age of sophisticated cyber attacks. *Computers & Security*, 72, 212 to 233. <https://doi.org/10.1016/j.cose.2017.09.001>